

Kubernetes — Documentation

Présentation

Là où Docker Compose suffit largement pour le développement local, la production d'ETNAir cible un cluster **Kubernetes**. Le passage à Kubernetes apporte trois bénéfices majeurs. D'abord, la **haute disponibilité** : plusieurs replicas de chaque service tournent en parallèle, et si un pod crash le service reste accessible grâce aux autres. Ensuite, le **scaling automatique** : un HorizontalPodAutoscaler ajuste le nombre de replicas en fonction de la charge réelle. Enfin, la **gestion déclarative** : tout l'état du cluster est décrit dans des manifests YAML versionnables, ce qui permet de reproduire l'environnement à l'identique sur n'importe quel cluster.

Les manifests sont rassemblés dans le dossier `k8s/` à la racine du repo. Le pipeline GitLab CI applique automatiquement ces manifests sur la branche `main` après avoir buildé et poussé les nouvelles images Docker sur Docker Hub.

Architecture sur le cluster

Un ingress controller (typiquement nginx-ingress) reçoit tout le trafic public sur le port 443. Il route les requêtes selon le chemin demandé : tout ce qui commence par `/api/` est envoyé vers le service Kubernetes `api`, et tout le reste est envoyé vers le service `frontend`. Le service `api` répartit la charge entre plusieurs pods (entre 2 et 10 selon la charge), tandis que le service `frontend` distribue vers deux pods statiques. Les deux pods d'API communiquent avec un service `postgres` qui pointe vers le pod PostgreSQL backé par un volume persistant. Enfin, un CronJob lance chaque nuit un job de backup qui dump la base et l'écrit sur un PVC dédié.

Inventaire des manifests

Le dossier `k8s/` contient une dizaine de fichiers YAML. Le fichier `postgres-deployment.yml` déclare le Deployment de PostgreSQL ainsi que son PersistentVolumeClaim. Le fichier `postgres-service.yml` crée le Service ClusterIP qui expose la base sur le port 5432 en interne (pas de NodePort, la base ne doit jamais être accessible directement depuis l'extérieur).

Le fichier `api-deployment.yml` déclare le Deployment de l'API avec ses replicas, ses ressources demandées (requests et limits CPU/mémoire), et ses probes de santé. Le fichier `api-service.yml` expose ce Deployment via un Service ClusterIP. Symétriquement, `frontend-deployment.yml` et `frontend-service.yml` font la même chose pour le frontend, avec un nginx qui sert le build statique généré par Vite.

Le fichier `ingress.yml` configure le routage public avec TLS terminé sur l'ingress (les certificats sont gérés par cert-manager et Let's Encrypt). Le fichier `hpa.yml` déclare le HorizontalPodAutoscaler qui pilote le scaling automatique de l'API. Le fichier `backup-cronjob.yml` déclare le CronJob de backup quotidien. Enfin `load-test.js` est un script k6 qui sert à valider la résilience du cluster sous charge.

Déploiement

Pour déployer manuellement l'ensemble des ressources sur un cluster, on lance simplement `kubectl apply -f k8s/`. Cette commande crée ou met à jour toutes les ressources définies dans le dossier. Pour vérifier que tout s'est bien passé, on consulte la liste des pods avec `kubectl get pods`, qui doit afficher un statut `Running` pour chacun. On peut aussi consulter les services avec `kubectl get svc`, et les autoscalers avec `kubectl get hpa`.

Pour suivre les logs d'un Deployment en temps réel, la commande est `kubectl logs -f deployment/api`. C'est particulièrement utile pour diagnostiquer un démarrage qui se passe mal. Si on pousse une nouvelle image Docker et qu'on veut forcer un redéploiement (sans changer le YAML), on utilise `kubectl rollout restart deployment/api`, ce qui crée de nouveaux pods avec la dernière image et termine progressivement les anciens (rolling update).

En CI/CD, le pipeline GitLab automatise tout cela. Sur chaque push sur `main`, il build les images Docker de l'API et du frontend, les pousse sur Docker Hub, puis applique les manifests Kubernetes via `kubectl apply -f k8s/`. Le résultat est qu'un simple `git push` suffit pour déployer une nouvelle version en production.

HorizontalPodAutoscaler

Le fichier `hpa.yml` configure l'autoscaling de l'API. La règle est simple : si la consommation CPU moyenne dépasse 70% pendant trois minutes, Kubernetes ajoute un nouveau pod, et continue jusqu'à un maximum de 10 replicas. Inversement, si la charge baisse durablement, des pods sont supprimés progressivement jusqu'au minimum de 2 replicas.

Le seuil de 70% est un compromis classique : un seuil plus bas (par exemple 50%) déclencherait du scaling plus tôt, mais consommerait plus de ressources pour pas grand-chose en cas de pics ponctuels. Un seuil plus haut (par exemple 90%) serait plus économe mais risquerait de saturer les pods avant que les nouveaux ne soient prêts.

Pour observer l'autoscaler en action, on peut lancer une session de monitoring continu avec `kubectl get hpa -w` et `kubectl get pods -w` dans deux terminaux séparés, puis déclencher un test de charge depuis un autre poste avec `k6 run k6/load-test.js`. On voit alors le CPU monter, les replicas augmenter progressivement jusqu'à atteindre un palier qui absorbe la charge, puis redescendre quelques minutes après la fin du test.

Gestion des secrets

Les variables sensibles ne sont jamais hardcodées dans les manifests YAML. Au lieu de cela, on crée un Secret Kubernetes qui contient `JWT_SECRET`, `POSTGRES_PASSWORD` et autres valeurs critiques :

```
kubectl create secret generic etnair-secrets \
  --from-literal=JWT_SECRET=<value> \
  --from-literal=POSTGRES_PASSWORD=<value>
```

Les Deployments référencent ensuite ces secrets via `secretKeyRef`, ce qui Kubernetes injecte la bonne valeur dans la variable d'environnement du conteneur au démarrage. L'avantage est que les secrets ne se retrouvent jamais dans le repo Git, ils restent stockés uniquement dans etcd côté cluster.

Pour une vraie production, on utiliserait plutôt un gestionnaire externe comme HashiCorp Vault ou Sealed Secrets, qui permettent de versionner les secrets de manière chiffrée. Pour ce projet pédagogique, la version simple suffit.

Stockage persistant

PostgreSQL nécessite un volume persistant pour ne pas perdre les données entre les restarts. Un `PersistentVolumeClaim` de 10 Gi en mode `ReadWriteOnce` est déclaré dans le manifest de la base. Kubernetes provisionne dynamiquement le PV correspondant via la storage class par défaut du cluster.

Pour les images uploadées, la situation est plus délicate car en production on a plusieurs pods d'API qui doivent tous voir les mêmes fichiers. La solution propre est soit un volume `ReadWriteMany` (par exemple NFS), soit la délégation

à un object store (S3, MinIO). Le fichier `k8s/minio-info.md` documente la configuration MinIO qui peut être utilisée comme alternative pour ce cas.

Probes de santé

L'API expose `/health` qui sert à deux types de probes Kubernetes. La `livenessProbe` vérifie périodiquement que le pod est vivant : si elle échoue trois fois de suite, Kubernetes redémarre le pod (en partant du principe qu'il est dans un état corrompu et qu'un redémarrage le réparera). La `readinessProbe` vérifie que le pod est prêt à recevoir du trafic : tant qu'elle échoue, le pod est retiré du service (pas tué, juste isolé). C'est utile au démarrage : pendant que le pod applique les migrations et fait son seed, il ne doit pas recevoir de trafic.

Les probes sont configurées avec des délais raisonnables : 30 secondes avant la première vérification (pour laisser le temps au pod de démarrer complètement), puis toutes les 10 secondes pour la liveness et toutes les 5 secondes pour la readiness.

Backups automatiques

Le CronJob défini dans `backup-cronjob.yml` s'exécute chaque nuit à 2h du matin (cron expression `0 2 * * *`). Il lance un conteneur PostgreSQL temporaire qui se connecte à la base de production, exécute `pg_dump` pour exporter toute la base, compresse le résultat avec gzip, et l'écrit sur un PVC dédié aux backups.

Le nommage des fichiers inclut la date du jour (`etnaïr-2026-06-04.sql.gz`), ce qui facilite la recherche d'un backup spécifique. Une politique de rotation manuelle est appliquée : les backups de plus de 30 jours sont supprimés à la main. Pour aller plus loin, on pourrait automatiser cette rotation avec un second CronJob qui supprime les vieux fichiers, ou déléguer l'archivage à un service cloud (S3 Glacier par exemple).

Monitoring

Le fichier `k8s/monitoring-info.md` décrit la stack de monitoring recommandée. Sa colonne vertébrale est **Prometheus** qui scrape les métriques de différentes sources : le kubelet pour les métriques système des pods, kube-state-metrics pour les métriques de l'API Kubernetes elle-même, et idéalement des métriques applicatives custom exposées par l'API via un endpoint `/metrics`.

Grafana est utilisé pour la visualisation, avec des dashboards qui montrent en un coup d'œil la santé du cluster : utilisation CPU/RAM des pods, taux de requêtes par seconde, latences (médiane, p95, p99), taux d'erreur HTTP 5xx, nombre de connexions actives PostgreSQL, taux de remplissage des PVC. **Loki** complète l'ensemble en agrégeant les logs de tous les pods dans une interface centralisée. Et **AlertManager** envoie des notifications (Slack, mail, PagerDuty) quand des seuils critiques sont franchis.

Les métriques applicatives les plus importantes à surveiller sur ETNAir sont la latence p95 de `/api/annonces` (le endpoint le plus chargé), le taux d'erreur 5xx, et la latence de la base de données.

Tests de charge avec k6

Le script `k6/load-test.js` simule un nombre configurable d'utilisateurs concurrents qui font des requêtes au frontend et à l'API. La configuration actuelle monte progressivement à 10 VUs (Virtual Users) sur 30 secondes, maintient ce niveau pendant 1 minute, puis redescend à 0 sur 30 secondes. Deux seuils sont définis : 95% des requêtes doivent répondre en moins de 500ms, et le taux d'erreurs doit rester inférieur à 1%.

Pour lancer un test de charge depuis sa machine, après avoir installé k6 (`choco install k6` sur Windows, `brew install k6` sur Mac), on exécute simplement `k6 run k6/load-test.js`. Pendant le test, on observe en parallèle le comportement du HPA avec `kubectl get hpa -w`. Si tout fonctionne correctement, on voit les replicas API monter sous l'effet de la charge, puis redescendre quelques minutes après la fin du test. C'est la démonstration concrète que la résilience automatique de l'infrastructure fonctionne.

Diagnostic des problèmes

Plusieurs symptômes reviennent souvent quand on déploie sur Kubernetes. Si un pod tombe en `CrashLoopBackOff`, c'est qu'il crash systématiquement au démarrage. On regarde alors les logs avec `kubectl logs <pod>` et la description complète avec `kubectl describe pod <pod>`. La cause la plus fréquente est une variable d'environnement manquante (souvent `DATABASE_URL` ou `JWT_SECRET`).

Si un service est injoignable depuis un autre pod, le coupable habituel est un mauvais sélecteur. On vérifie les endpoints du service avec `kubectl get endpoints <service>` : s'ils sont vides, c'est que les labels des pods ne matchent pas le sélecteur du service.

Si le HPA affiche `unknown` pour le CPU, c'est que `metrics-server` n'est pas installé sur le cluster. On peut le déployer avec un simple `kubectl apply -f https://github.com/kubernetes-sigs/metrics-server/...`. Si un pod reste en `Pending`, c'est généralement qu'aucun nœud n'a assez de ressources libres pour l'accueillir, ou qu'il demande un volume qui ne peut pas être provisionné.

Enfin, si l'ingress retourne des 502, c'est souvent que la `readinessProbe` du backend échoue : le service est techniquement existant mais aucun pod n'est marqué prêt à recevoir du trafic.